



The CS Plan

NUS Orbital 2026

Milestone 3

Group 6640

Chester Lim

Ian Sin

Deployment

Technical Proof-of-Concept: <https://thecsplan.vercel.app> **DOES NOT WORK ON MOBILE

Proposed Level of Achievement

Apollo

Aim

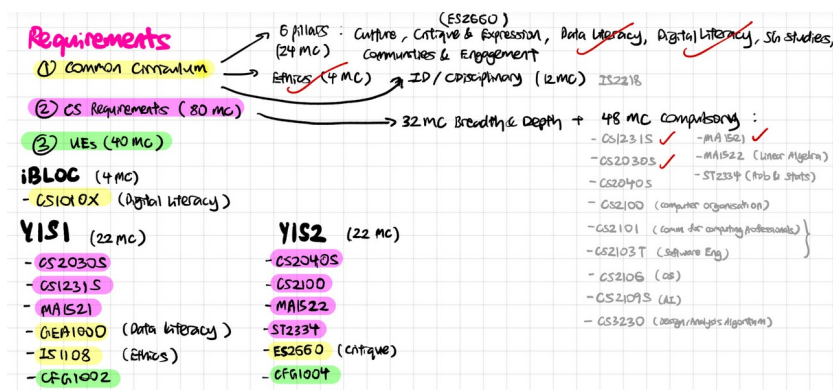
The CS Plan aims to become the **academic planning and tracking companion** for NUS Computing students. We hope to help students plan modules across their entire duration of study, and see how the current plan contributes to their degree requirements, making planning decisions with less manual cross-checking. We also hope to help students track their pre/post S/U GPA, S/U usage, and simulate what grades are required to hit a target GPA.

Milestone 1 is focused on the planning feature for the Computer Science single degree for AY 2025/26. This narrow first target is intentional: it allows us to demonstrate the technical proof-of-concept for the planner, user profile, requirement rules engine, and requirement evaluation workflow. Once successful, we can continue to add the GPA feature, and afterwards scale to more double-major combinations, cohorts, degrees, and faculties.

Our goal is **convenience and clarity** in academic planning and tracking for NUS students.

Problem Motivation

At the start of each academic year or semester, students toggle between multiple tools while planning their modules: NUSMods, the NUS Computing degree requirement pages, personal notes, spreadsheets, and calculators. This is a frustrating process that students have to endure, especially when the degree requirements are just a static table of rules and number of units to clear. The problem becomes even harder when they include internships, student exchange, focus area choices, unrestricted electives, and many more factors that increase the complexity of module planning.



The above screenshot is how we planned our Year 1 modules before each semester started.

Students are not only choosing modules, but also checking whether their choices satisfy curriculum pillars, Inter/Cross-Disciplinary counts, degree foundation units, breadth and depth rules, and more requirements. Module planning is very dynamic; any addition, removal or reordering of a module can affect the fulfillment of degree requirements. Hence, a static planner that most students currently use is ineffective, with them often needing to resort to manual calculations.

On top of this, students need to plan their modules to optimise their GPA. It is troublesome to track which modules have been S/U-ed and to keep recalculating pre and post S/U GPA, with students again resorting to a calculator, or an Excel spreadsheet at best.



CS1010X - A+
- Midterm 27/32, PE 32/32
Y1S1
CS2030S - A
- Midterm 30/40, PE1 29/40, PE2 37/40
- Finals 45.5/60, Total 81.75
CS1231S - A
- Midterm 43/50
- Finals 70/100, Total 81
MA1521 - A
GEA1000 - A+
IS1108 - B+ - SU
- Midterm 19/30
- Total 81/100
CFG1002 - CS
Y1S2
CS2040S
CS2100
- Midterm 23.5/40
MA1522
ST2334
- Midterm 17/20
ES2660
- CA1 80.7
CFG1004
TOTAL
Pre SU: 4.83, Post SU: 5.00

The above screenshot of our Notes app is how we tracked our GPA and grades; all manually.

For Milestone 1, The CS Plan addresses the first half of the identified problem for the Computer Science single degree. It combines module planning and degree requirement progress in one unified dashboard. The planner can record completed, current, and future modules all the way up to graduation. Placeholders allow students to reserve space for unknown choices such as a future UE or ID/CD module. Upon any change in the planner, the webapp evaluates the most recent plan against the degree requirements, and the results are displayed as progress bars for each requirement to the user. This eliminates the need to check what requirement a module fulfills, and manually count units.

The upcoming GPA tracking feature will address the second half of the motivation. It will allow students to track grades, S/U usage, pre/post S/U GPA, and simulate what future grades are required to hit a target GPA. This is planned for the next milestone after the foundation planner and rules engine are stable.

User Stories

1. As a student with a Gmail account, I want to sign in conveniently with my Google account so that my plan is saved securely and available whenever I return.
2. As a first-time user, I want to enter my degree, cohort, starting semester, current semester and expected graduation semester so that the app creates the right planning timeline.
3. As a student who is planning modules, I want to search for modules by code or title and add them into the correct semester so that I can build a complete academic plan. I also want advisory warnings when I have not cleared all the necessary prerequisites for a module so that I can revise my plan accordingly.
4. As a student who has not decided every module yet, I want to add placeholders such as UE, ID, CD, or CS Foundation so that my plan can reserve units and contribute to the respective degree requirement before I have decided on the exact module.
5. As a student checking degree requirement progress, I want the webapp to evaluate my current plan against the requirement rules so that I know which requirements are fulfilled, partial, or missing. I also want to view the requirement rule definitions on another page separate from my plan, just like on the NUS curriculum website.
6. As a returning user, I want to update my academic settings safely so that a changed starting, current or graduation semester resizes my plan without deleting existing module choices.
7. As a student who is concerned with my GPA and S/Us, I want to be able to track my pre and post S/U GPA, and what modules I have S/Ued. I also want to simulate future grades to hit a target GPA, so that degree planning and performance planning can happen in the same workflow.
8. As a student who is planning my academic journey, I want to add and customize multiple plans. For example, the scenario where I am doing an internship in Y3S1, and that in Y3S2 are two different plans with different modules in each semester. I want to toggle between different plans and view respective GPAs for each plan.
9. As a student who wants to share plans with his peers and draw inspiration from other plans, I want to export my own plan and import other's plans into a new plan. I want to be able to do this conveniently via a copyable link or payload. I want to choose whether I want to include my GPA data in the payload when exporting.

Future Milestone

10. As an admin, I want to update requirement sets, rule definitions, and module tags through a dashboard so that curriculum data can be maintained safely without manually seeding the database or editing application code.

11. As an admin, I want to copy an existing ruleset from one cohort to another when the requirements are identical so that AY2026/27 can be created from AY2025/26 quickly and with fewer manual errors.
12. As a Business Analytics or Business Artificial Intelligence Systems student, I want to select my own degree and cohort so that the planner evaluates my modules against the correct requirement set.
13. As a double major or double degree student, I want to choose my course as a standalone programme option so that the app follows the published structure of that course instead of loosely combining two separate majors.
14. As a maintainer, I want requirement changes to be validated against sample plans before publishing so that scaling to more degrees does not break existing CS requirement evaluation.

Future Plans

As an NUS student outside the School of Computing, I want to choose my home faculty, programme, and cohort so that the planner evaluates my academic plan against the correct faculty-owned ruleset.

As a faculty curriculum maintainer, I want curriculum drafts to record their source, reviewer, approval history, and rollback target so that published requirement changes are traceable and recoverable.

As a student with unfilled semesters, I want the planner to recommend modules that satisfy my remaining requirements while respecting prerequisites, module availability, timetable clashes, and workload limits so that I can turn degree progress into a feasible semester plan.

Core Features

1. Login using Google OAuth and Onboarding

Description

The application uses Google OAuth as the only sign-in method. The login page sends the browser to the backend OAuth route. The backend builds the Google authorization URL, exchanges the callback code for Google user information, upserts a local user record, and sets an HTTP-only session cookie. After login, the frontend calls `/api/me` to determine whether the user already has an academic profile.

If no profile exists, the user is redirected to onboarding. The onboarding form collects the programme, cohort, starting semester, and expected graduation semester. For Milestone 1, the visible programme is Computer Science and the supported cohort is AY2025/26. When the profile is saved, the backend creates a Primary Plan with semester columns from the selected starting semester to the graduation semester.

The image displays two side-by-side screenshots of a web application interface. The left screenshot shows a login page titled 'The CS Plan' with a subtitle 'Your NUS Computing Planning Companion'. It features a 'Continue with Google' button and a note: 'Uses Google Sign-in only. Your module plan is stored separately from Google and can be exported later.' The right screenshot shows an onboarding screen titled 'Set up your degree plan'. It includes a note: 'Milestone 1 only supports Computer Science single degree for AY25/26.' Below this are five dropdown menus for 'Programme' (selected: Computer Science), 'Cohort' (selected: AY2025/26), 'Starting Semester' (selected: Y1S1 · Year 1 Semester 1), 'Current Semester' (selected: Y1S1 · Year 1 Semester 1), and 'Expected Graduation Semester' (selected: Y4S2 · Year 4 Semester 2). At the bottom is a 'Create planner' button.

Left: Login page // Right: Onboarding screen for new user

Implementation Philosophy

Authentication is kept separate from academic state. Google proves identity, but the application stores its own User and StudentProfile records. This avoids coupling degree planning data to Google tokens or account internals. The session cookie is HTTP-only so client-side code does not need to store the JWT directly.

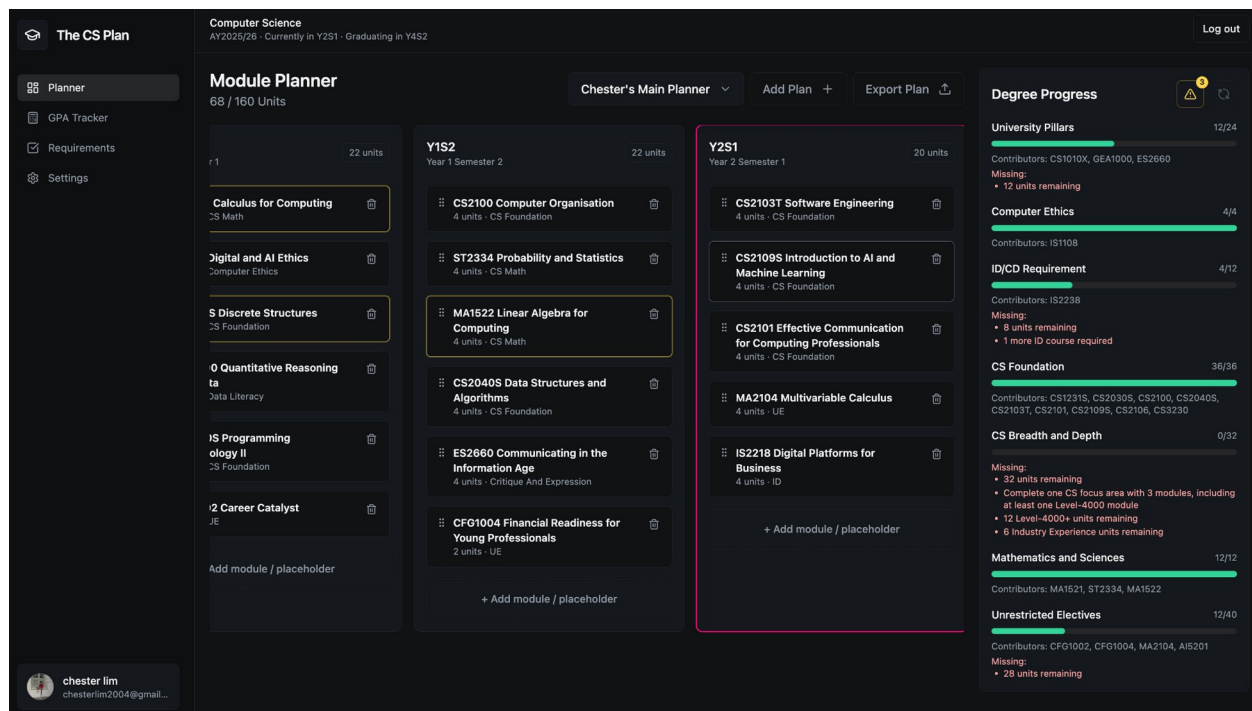
Onboarding is treated as a gate before the main dashboard. The planner and requirements screens need the user's programme and cohort to know which requirement set to evaluate. Redirecting users without a profile to onboarding prevents the app from evaluating a plan with missing academic context.

Important file	Purpose
apps/web/src/pages/LoginPage.tsx	Shows the Google sign-in entry point and routes to the backend login URL.
apps/api/src/routes/authRoutes.ts	Defines <code>/api/auth/google</code> , <code>/api/auth/google/callback</code> , and <code>logout</code> .
apps/api/src/services/googleAuthService.ts	Builds the Google OAuth URL and exchanges the callback code for user info.
apps/api/src/middleware/auth.ts	Creates, sets, clears, and verifies the <code>tcp_session</code> cookie.
apps/web/src/pages/OnboardingPage.tsx	Collects programme, cohort, starting semester, and graduation semester.
apps/api/src/services/profileService.ts	Creates or updates the StudentProfile and reconciles plan semesters.

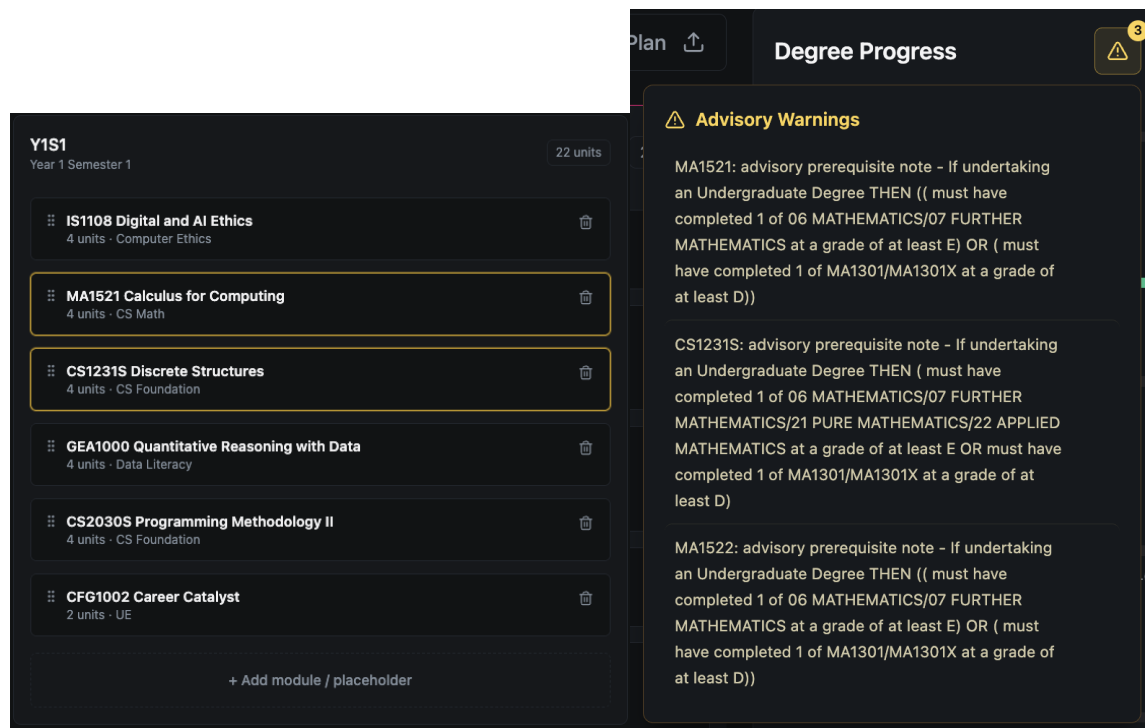
2. Module Planner, NUSMods Module Data, Placeholders, and Prerequisite Advisory Warnings

Description

The planner is the main workspace. It displays semester columns based on the user profile and lets the user add real modules or placeholders. Real modules come from the module catalogue. The seed process imports module information from the NUSMods API, including module code, title, units, department, faculty, description, prerequisite text, and prerequisite trees when available. If the external import is unavailable, local fallback module data keeps the app usable for development.

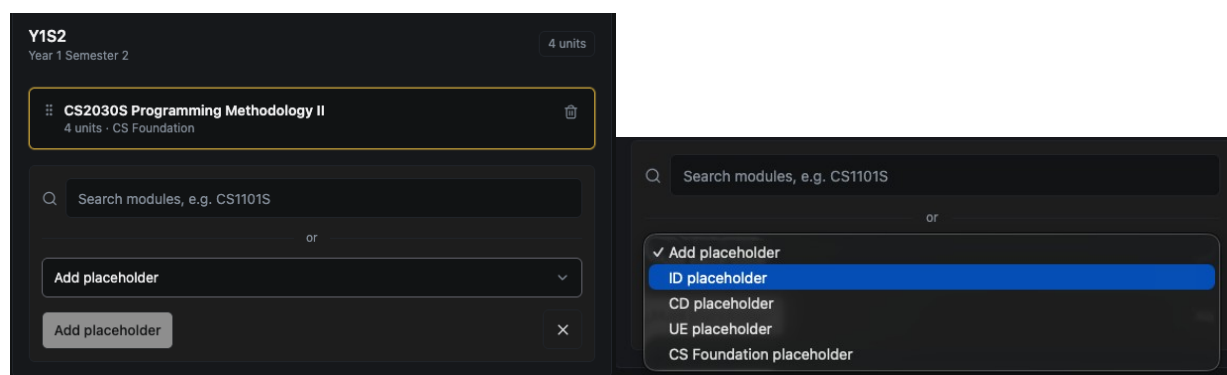


Homepage of The CS Plan

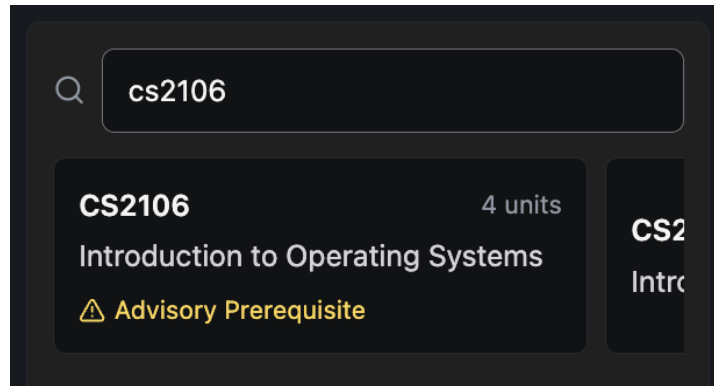


Left: Interface for each semester // Right: Prerequisite advisory warnings which appear as a notification badge at first and expand into warnings when clicked, if the prerequisites for a module are not fulfilled.

Module search queries the backend by module code or title. The frontend shows the search results in the add panel and inserts the chosen module into the selected semester. Users can also add placeholders such as ID, CD, UE, and CS Foundation. These placeholders are important because students often know that a requirement slot must exist before they know the exact module that will fill it.



Left: “Add Module” interface: module search and placeholders // Right: Placeholder dropdown



Horizontal-scroll search results for a module code query. Users can also search partial strings or descriptions.

The planner supports drag-and-drop movement between semesters, removal of items, total unit counting, and cached requirement evaluation. When the plan changes, the updated plan is saved through the plan API. The app then refreshes the evaluation and shows warnings, including unknown module warnings from the evaluator and prerequisite advisory warnings from the rules-engine package.

Implementation Philosophy

The planner stores plan items as a discriminated union: module items and placeholder items. This is cleaner than treating placeholders as fake modules. Real modules have moduleCode, units, and status. Placeholders have requirementId, label, and units. That difference matters because a placeholder should count toward a requirement category during planning, but it should not pretend to be a real NUS module.

NUSMods import is implemented as a backend seed concern rather than a frontend scrape. The backend normalises external module data into the shared Module schema and stores it in MongoDB. The UI can then search a stable local catalogue and does not need to understand NUSMods response details.

Prerequisite warnings are advisory, not blocking. Students may have exemptions, special approvals, or equivalent prior knowledge. The app therefore warns when prerequisites appear unsatisfied but still allows the plan to be saved. This matches the purpose of a planning assistant rather than a registration system.

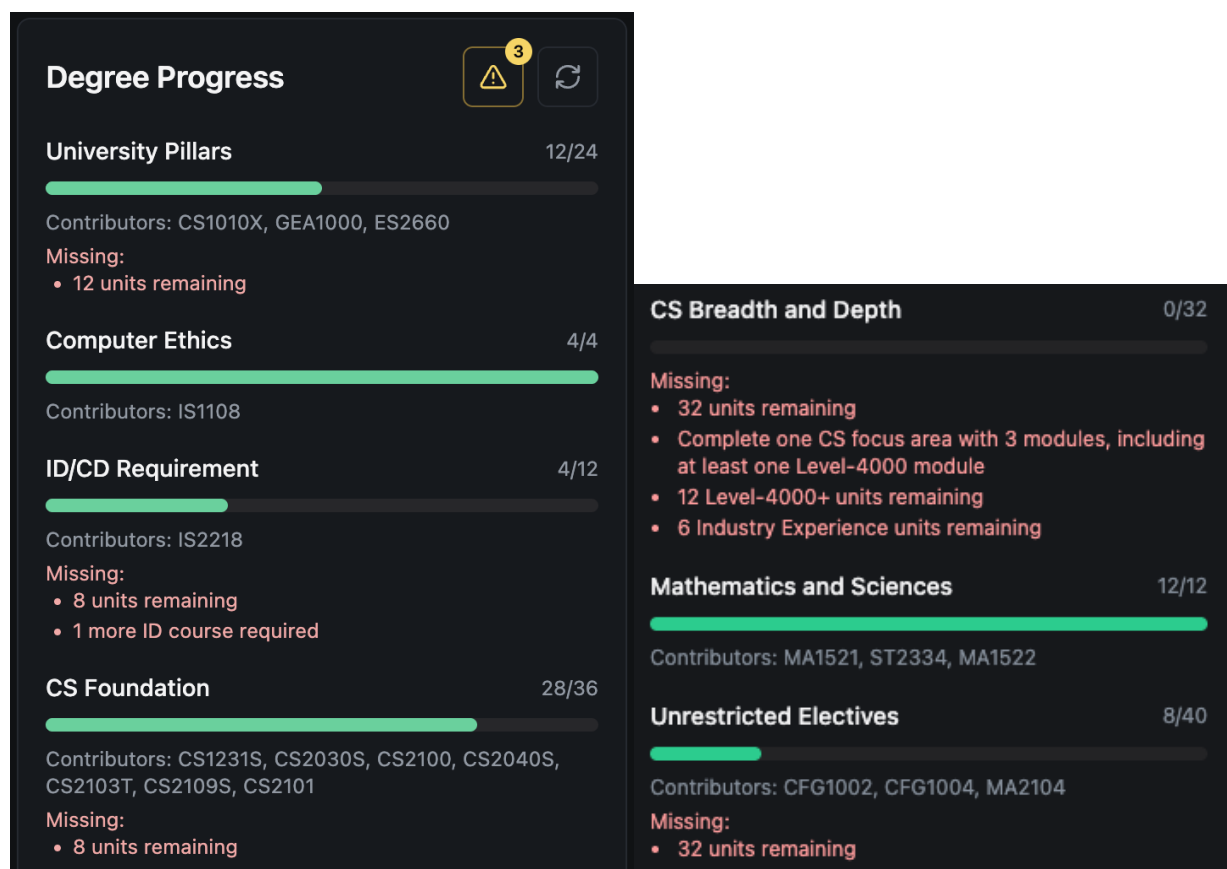
Important file	Purpose
apps/web/src/pages/PlannerPage.tsx	Main planner UI, search panel, placeholders, drag/drop movement, evaluation cache, warnings.
apps/api/src/services/nusmodsImportService.ts	Fetches and normalises NUSMods module data with bounded concurrency.
apps/api/src/seed.ts	Seeds modules, module requirement tags, and requirement sets into MongoDB.
apps/api/src/services/moduleService.ts	Searches modules by code/title and returns individual module details.
apps/api/src/services/planService.ts	Creates, lists, updates, deletes, exports, imports, and evaluates owner-scoped plans.
packages/shared/src/index.ts	Defines PlanItem, ModulePlanItem, PlaceholderPlanItem, SemesterPlan, and Plan schemas.
packages/rules-engine/src/evaluatePlan.ts	Provides getPrerequisiteWarnings and evaluator warnings used by the planner.

3. Degree Requirement Fulfillment

Description

Degree progress is calculated by evaluating the saved plan against a versioned requirement set. The backend gets the owned plan, all modules, the requirement set for the plan's programme and cohort, and the module requirement tag lookup. It then calls `evaluatePlan` from the rules-engine package and returns requirement progress rows to the frontend.

Each progress row includes the requirement id, label, completed units, required units, percentage, status, contributors, missing items, and warnings. The status is fulfilled, partial, or missing. This makes the result usable both for a summary dashboard and for detailed requirement pages.



The Degree Requirement Progress for a sample academic plan, located on the right of the plan.

The current CS requirement rules include University Pillars, Computer Ethics, ID/CD, CS Foundation, CS Breadth and Depth, Mathematics and Sciences, and Unrestricted Electives. The rules support simple module lists, units from tags, capped units from tags, combined units, structured ID/CD constraints, structured breadth and depth constraints, placeholders, and residual units.

Implementation Philosophy

The requirement ruleset is an independent rules engine so the application does not hardcode degree conditionals across React components or Express routes. Requirements are represented

as data in versioned JSON-like rule objects. The evaluator is a pure TypeScript package that accepts a requirement set, a plan, modules, and module requirement tags. This makes it easier to add future cohorts or programmes by adding data and extending rule types deliberately, rather than scattering if-else logic across the app.

The moduleRequirementTags model exists because the same module can mean different things for different degrees or cohorts. For example, a module might be treated as a CS breadth and depth module for one programme, an unrestricted elective for another, or carry a different requirement tag in a later cohort. Storing tags with programme and cohort keeps module identity separate from curriculum interpretation.

The evaluator also tracks allocation. Once a module has been claimed by a requirement, it is not double-counted in another non-residual requirement. Overflow modules can flow into residual units such as unrestricted electives. This mirrors how students reason about degree planning: a module should satisfy the most specific relevant bucket first, then excess units can be counted where allowed.

Important file	Purpose
packages/rules-engine/src/evaluatePlan.ts	Pure evaluator and prerequisite warning logic.
packages/data/src/computerScienceData.ts	CS requirement set, focus areas, generated tags, and fallback modules.
apps/api/src/services/requirementService.ts	Loads the latest RequirementSet from MongoDB or falls back to seeded data.
apps/api/src/services/moduleRequirementTagService.ts	Returns programme/cohort-specific tag mappings and lookup maps.
apps/api/src/services/planService.ts	Composes plan, modules, tags, and rules before calling evaluatePlan.
packages/rules-engine/src/tests/evaluatePlan.test.ts	Unit tests for the evaluator behaviour.

4. Degree Requirements Page and Settings Page

Description

The Degree Requirements page displays the rules behind the evaluation. Instead of only showing progress, it explains what each requirement expects: required units, accepted tags, placeholder support, ID/CD limits, breadth and depth focus area rules, Level-4000 requirements, industry experience constraints, and unrestricted elective behaviour. This gives the user an easily accessible readable reference.

Computer Science
AY2025/26 · Graduating in Y4S2

Log out

Degree Requirements
Computer Science, AY2025/26

University Pillars
24 units
university-pillars

- Complete 24 units across the six university pillars.
- Only one 4-unit course counts for each pillar.
- Pillars: Digital Literacy, Critique And Expression, Cultures And Connections, Data Literacy, Singapore Studies, Communities And Engagement.

ID/CD Requirement
12 units
idcd

- Complete 12 units from interdisciplinary and Cross-Disciplinary courses.
- Take at least 2 ID courses.
- Count no more than 1 CD course.

CS Breadth and Depth
32 units
cs-breadth-and-depth

- Complete 32 units of CS Breadth and Depth modules.
- Complete one CS focus area with 3 focus-area modules, including at least 1 Level-4000 module.
- Complete at least 12 units at Level-4000 or above.
- Complete 6 to 12 units of Industry Experience, or use CP4101 as the dissertation replacement.
- Non-industry modules must be CS/IFS/CP-coded.
- At most 12 units of non-industry CP-coded modules may count.

Computer Ethics
4 units
computer-ethics

- Complete 4 units from modules tagged Computer Ethics.

CS Foundation
36 units
cs-foundation

- Complete 36 units from modules tagged CS Foundation.
- Matching placeholders can be used for planning: CS Foundation.

Mathematics and Sciences
12 units
cs-math

- Complete 12 units from modules tagged CS Math.

The Degree Requirements page, a readable reference for the user's degree requirement rules.

The Settings page lets users edit their academic profile. Changing the profile affects the requirement rules used by the account and resizes saved plans. The UI asks for confirmation before saving because a profile change can affect every saved plan. On confirmation, all of the user's plans are evaluated against the new requirement rules, so that degree progress data is updated.

Settings

Academic Profile

Changing these settings updates the degree requirement rules used by your account and resizes every saved plan.

Degree

Computer Science

Cohort

AY2025/26

Starting Semester

iBLOC

Current Semester

Y2S1 · Year 2 Semester 1

Graduation Semester

Y4S2 · Year 4 Semester 2

Save Academic Profile

The Settings Page.

Implementation Philosophy

The requirements page is intentionally separate from the planner. The planner is for action; the requirements page is for explanation. Keeping them separate avoids crowding the planning workspace while still making the rules transparent.

Settings updates reuse the onboarding profile service instead of having a separate update path. This means the same schema validation and semester reconciliation logic applies whether the user is creating the profile for the first time or changing it later. Existing semester items are preserved when possible, which protects user data during plan resizing.

Important file	Purpose
apps/web/src/pages/RequirementsPage.tsx	Renders readable descriptions for each requirement rule type.
apps/web/src/pages/SettingsPage.tsx	Lets users update degree, cohort, starting semester, and graduation semester.
apps/api/src/routes/requirementRoutes.ts	Serves requirement sets by programme and cohort.
apps/api/src/routes/profileRoutes.ts	Serves and updates the authenticated user's academic profile.
apps/api/src/services/profileService.ts	Reconciles plan semesters after academic profile changes.

5. GPA and S/U Tracker

Description

The GPA tracker connects academic performance tracking to the same saved plan used by the module planner. Instead of making students maintain a separate spreadsheet, each planned module or placeholder can store a grade and an S/U flag. The GPA page then reads the active plan, groups items by semester, and calculates both current GPA and entire-plan GPA.

The tracker distinguishes between pre-S/U and post-S/U GPA. Pre-S/U GPA counts all graded modules before S/U removal, while post-S/U GPA excludes modules that have been marked as S/U or have S/U-only grades. This makes it possible for students to see the academic effect of S/U decisions without recalculating everything manually.

The screenshot displays the GPA Tracker interface. At the top, it shows 'GPA Tracker' with a subtitle 'Assign grades to modules in your plan. Checked S/U modules and CS grades do not affect GPA.' and a button 'Chester's Main Planner'. A yellow badge indicates 'S/Us used: 3 / 8'. The main section is divided into three columns: GPA, Pre-S/U, and Post-S/U. The GPA column shows 'Current GPA' (Before Y2S1) as 4.68 and 'Entire Plan GPA' (All planned semesters) as 4.70. The Pre-S/U column shows 'Current GPA' as 4.68 and 'Entire Plan GPA' as 4.70. The Post-S/U column shows 'Current GPA' as 4.89 and 'Entire Plan GPA' as 4.92. Below these are four semester cards: Y1S1, Y1S2, Y2S1, and Y2S2. Each card lists modules with their grades and S/U status. The Y2S1 card is highlighted with a red border, showing a semester GPA of 5.00. The Y2S2 card shows a semester GPA of 4.75.

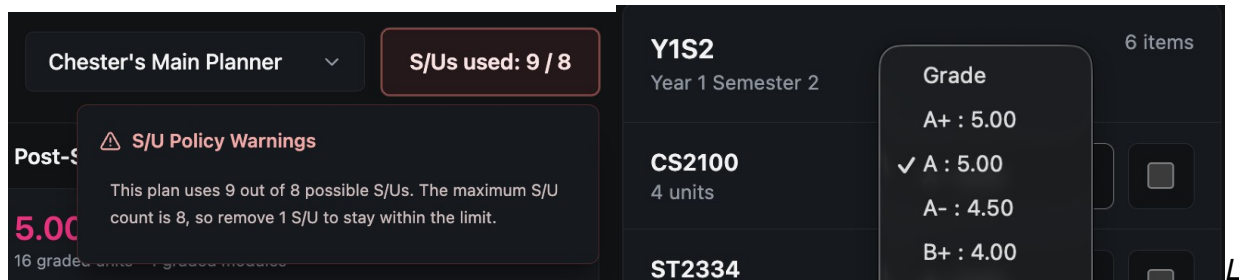
GPA	Pre-S/U	Post-S/U
Current GPA Before Y2S1	4.68 44 graded units - 11 graded modules	4.89 36 graded units - 9 graded modules
Entire Plan GPA All planned semesters	4.70 60 graded units - 15 graded modules	4.92 48 graded units - 12 graded modules

Y1S1 Year 1 Semester 1	Y1S2 Year 1 Semester 2	Y2S1 Year 2 Semester 1	Y2S2 Year 2 Semester 2
MA1521 4 units A+ : 5.00	CS2100 4 units A : 5.00	CS2103T 4 units A : 5.00	CS2109S 4 units A : 5.00
IS1108 4 units B+ : 4.00	ST2334 4 units A : 5.00	CS2101 4 units B+ : 4.00	MA2104 4 units A : 5.00
CS1231S 4 units A : 5.00	MA1522 4 units A- : 4.50	IS2218 4 units Grade	
GEA1000 4 units A+ : 5.00	CS2040S 4 units A- : 4.50		
CS2030S 4 units A : 5.00	ES2660 4 units B : 3.50		
CF61002 2 units CS : -	CF61004 2 units CS : -		
SEMESTER GPA 16 graded units - 4 graded modules 5.00	SEMESTER GPA 16 graded units - 4 graded modules 4.75	SEMESTER GPA 12 graded units - 3 graded modules 5.00	

GPA tracker dashboard showing pre-S/U and post-S/U summaries for the selected plan, and the total number of S/Us used.

Each semester card lists the modules and placeholders in that semester, exposes a grade dropdown, includes an S/U checkbox, and displays the semester GPA. The current semester is highlighted using the currentSemester stored in the student profile, so the page is tied to the user's academic timeline rather than a fixed year assumption.

The S/U counter also reflects the application's S/U policy logic by tracking how many modules have been marked as S/U and warning the user when the planned usage goes beyond the allowed limit, or violates NUS' S/U policies. For example, no more than 3 S/Us after Y2S2 for the AY25/26 cohort onwards. This keeps the GPA tracker useful not only as a calculator, but also as an advisory tool for checking whether S/U decisions remain within policy constraints.



Left: S/U Policy Warnings // Right: Per-semester GPA cards where students can enter grades and mark modules as S/U.

Implementation Philosophy

The implementation keeps grade data inside PlanItem because the grade belongs to a specific planning scenario. If a student creates two different plans, each plan can carry different simulated grades and S/U choices. This is more flexible than storing grades globally per module code, because the same module may appear in different academic scenarios.

The GPA calculation is intentionally deterministic and local to the frontend view. The shared package defines the allowed grades and grade points, while the GPA page applies the calculation rules consistently across semester, current, and whole-plan views. The backend still validates and persists the plan through the same Plan schema, so grade changes use the existing owner-scoped plan update flow.

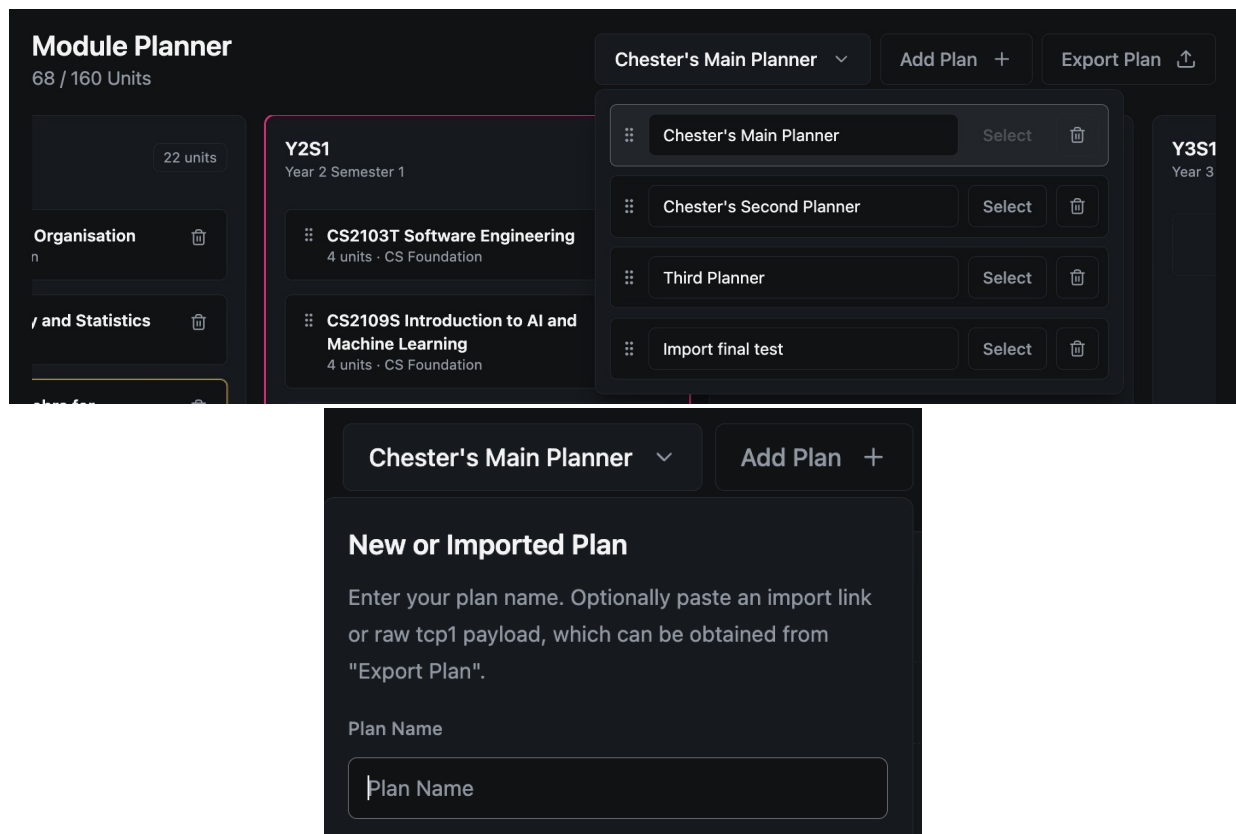
Important file	Purpose
apps/web/src/pages/GpaPage.tsx	Main GPA and S/U tracker UI, grade dropdowns, S/U checkboxes, current/entire plan summaries, and per-semester GPA calculation.
packages/shared/src/index.ts	Defines ModuleGrade, gradePoints, and grade/isSu fields on module and placeholder plan items.
apps/api/src/models/Plan.ts	Persists semesters and mixed plan items, including grade and S/U fields, as part of the owner-scoped plan document.
apps/api/src/services/planService.ts	Validates and saves updated plans through PlanSchema before writing changes to MongoDB.
apps/web/src/lib/api.ts	Provides the updatePlan and listPlans calls used by the GPA tracker.

6. Multiple Plans Feature

Description

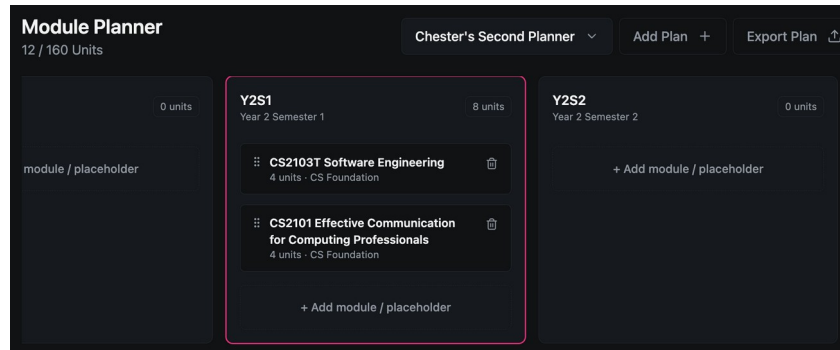
The multiple plans feature lets a student maintain more than one academic scenario under the same account. This supports planning situations where a student wants to compare different module choices, exchange semesters, internship timing, workload distribution, or graduation timelines without overwriting the main plan.

Plans can be created, selected, renamed, reordered, and deleted from the planner. A newly created plan uses the student profile's programme, cohort, starting semester, and graduation semester to build the correct semester range. The profile stores `primaryPlanId` and `planOrder` so the application knows which plan should open by default and how the plan menu should be displayed.



Multiple plan (and Add Plan) controls allow students to maintain alternative academic scenarios under one account.

Deletion is guarded so a user cannot remove the final remaining plan. When the selected plan changes, the planner, requirement progress, warning cache, and GPA page all follow the selected or primary plan. This keeps the rest of the application plan-aware without duplicating separate flows for each page.



Each plan has its own semester items while sharing the same user profile and programme context. The GPA page will read the active plan and do calculations for that plan. The plan can be selected from within the GPA page too.

Implementation Philosophy

Multiple plans are implemented as real Plan documents instead of temporary frontend-only copies. This makes each scenario persistent, owner-scoped, and available after login. StudentProfile stores ordering and primary-plan metadata, while the Plan collection stores the full semester layout.

Plan creation and profile updates are coordinated carefully. After a plan is created, the frontend immediately updates the profile so the new plan becomes the primary plan and appears in the plan order. Profile reconciliation still resizes all owned plans when the user changes starting or graduation semester, preserving existing items for semesters that remain valid.

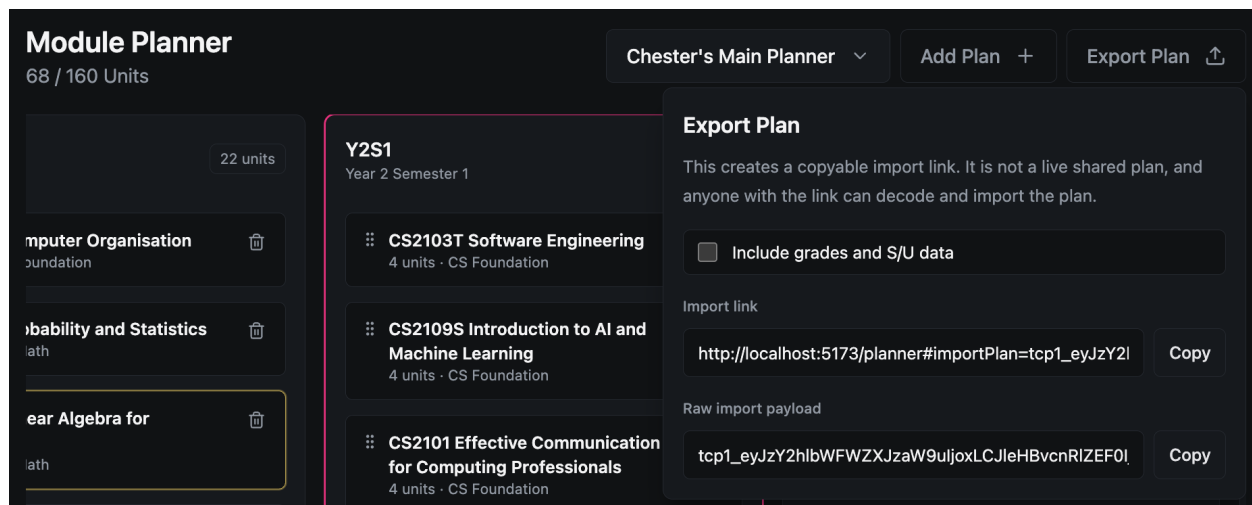
Important file	Purpose
apps/web/src/pages/PlannerPage.tsx	Plan menu, create/import flows, rename behaviour, drag ordering, deletion guard, and primary plan selection.
apps/web/src/pages/GpaPage.tsx	Uses planOrder and primaryPlanId to select the GPA plan consistently with the planner.
apps/api/src/services/profileService.ts	Maintains primaryPlanId, sanitises planOrder, and reconciles semesters across owned plans after profile changes.
apps/api/src/services/planService.ts	Creates, lists, updates, deletes, and serializes owner-scoped Plan documents.
apps/api/src/models/StudentProfile.ts	Stores primaryPlanId and planOrder references to Plan.
packages/shared/src/index.ts	Defines Plan and StudentProfile schemas used by both frontend and backend.

7. Plan Import / Export via Import Link or tcp Payload

Description

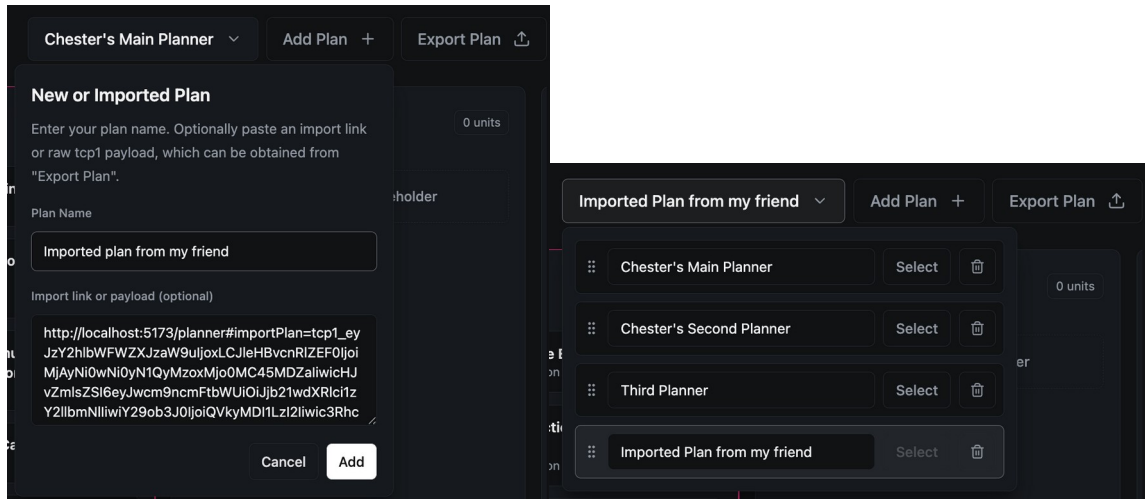
Plan import and export are built for plan sharing, matching the user story where students want to share plans with peers and draw inspiration from other plans. A student can export the active plan as a schema-versioned payload, copy a raw tcp1_ payload, or copy an import link that embeds the payload in the planner URL hash.

The exported snapshot contains schemaVersion, exportedAt, a StudentProfile snapshot, and a Plan snapshot with MongoDB ids removed. The frontend encodes this JSON snapshot into a compact base64url tcp1_ payload. During export, users can choose whether grades should be included. By default, grades and S/U flags are stripped before sharing so academic performance data is not accidentally shared with the plan structure.



Plan export modal producing a shareable link or tcp payload for peer-to-peer plan sharing, with the option to include grades during export.

Import accepts two forms: a full import link or raw tcp1_ payload. The frontend decodes the payload and sends the structured PlanExport object to the backend. The backend validates the payload using PlanExportSchema and creates a new user-owned plan named as an imported copy. The imported plan is then added to the user's plan order and selected as the primary plan.



Imported plans become separate editable plans rather than overwriting the sender's or receiver's existing plan.

Implementation Philosophy

The sharing format is a snapshot, not a live shared document. This prevents accidental cross-account writes and keeps ownership simple: once imported, the receiving user owns a separate plan. Removing database ids from the exported plan also prevents the import flow from depending on another user's MongoDB records.

The tcp1_ prefix acts as a lightweight schema marker. It makes pasted payloads identifiable and leaves room for future payload versions if the export schema changes. The backend remains the final authority because import still passes through Zod validation and createPlan, ensuring the imported plan has the same ownership and persistence rules as manually created plans.

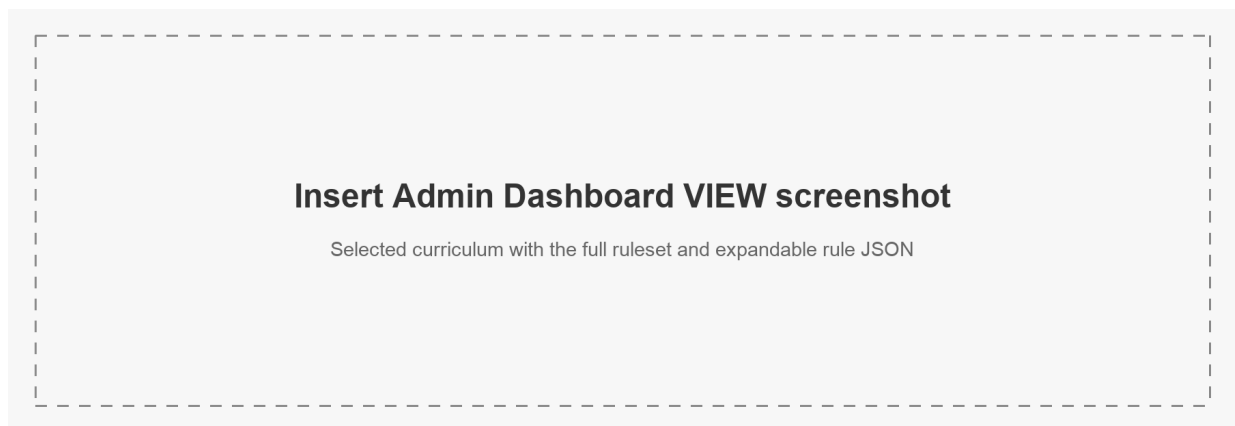
Important file	Purpose
apps/web/src/pages/PlannerPage.tsx	Encodes tcp1_ payloads, extracts import payloads from links, strips grades by default, and wires export/import UI.
apps/api/src/routes/planRoutes.ts	Exposes /api/plans/:planId/export and /api/plans/import behind authentication.
apps/api/src/services/planService.ts	Builds PlanExport snapshots, removes MongoDB ids on export, validates imports, and creates imported plans.
packages/shared/src/index.ts	Defines PlanExportSchema and shared profile/plan schemas used to validate exported payloads.
apps/web/src/lib/api.ts	Provides exportPlan and importPlan API calls to the planner.

8. Admin Dashboard: View Curriculum Rules and Module Mappings

Description

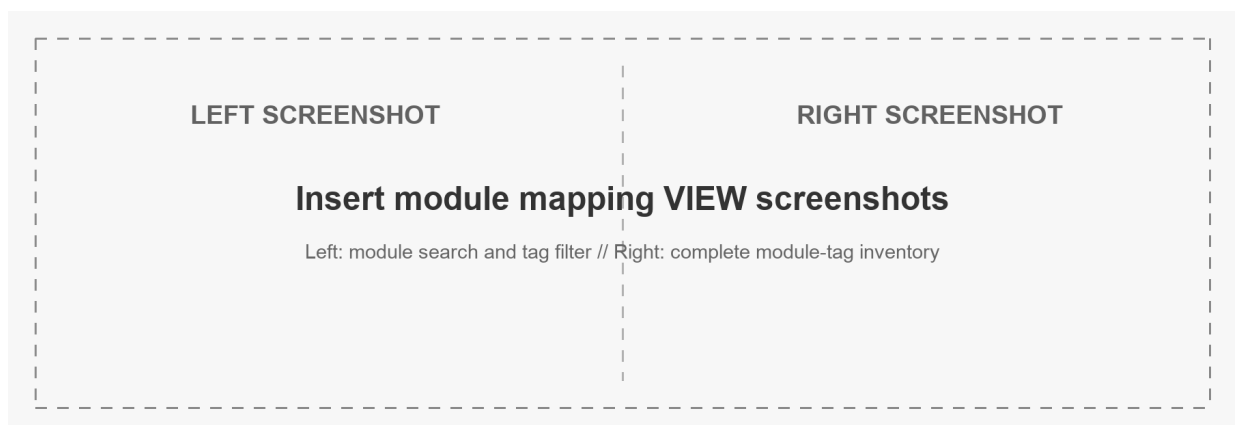
The administrator dashboard is a role-protected workspace for inspecting every published curriculum. An administrator first selects a programme and cohort from the curriculum catalogue. Each catalogue entry shows the latest version and opens the complete RequirementSet for that curriculum, including its total unit target, source note, ordered requirement rules, and current version number. This makes the database representation visible in the web application instead of requiring direct MongoDB inspection or reading seeded TypeScript data.

Every requirement is displayed as a rule card and can be expanded to show the rule's complete JSON. This is important because the rule type alone is not enough to understand a curriculum: the administrator must also see required units, accepted tags, module choices, tag caps, placeholder categories, focus areas, industry constraints, honours pathways, and any advisory text. Viewing the canonical JSON keeps the dashboard faithful to the same RequirementRule shape that is evaluated by the rules engine.



Admin dashboard curriculum view showing the selected programme, cohort, latest version, total units, source note, ordered rules, and expandable rule JSON.

The Module Tags tab exposes the entire programme-and-cohort-specific module mapping. Without a query, it pages through all mapped modules; with a query, it searches the module catalogue by module code or title and joins the matching modules to their current tags. Administrators can also filter the results by one tag. A separate tag inventory lists every tag used by the curriculum together with the modules mapped to it, allowing the administrator to inspect the classification vocabulary before editing a rule or mapping.



Left: searchable module mappings with a module-tag filter // Right: the complete tag inventory for the selected curriculum.

Important file	Purpose
apps/web/src/pages/AdministratorPage.tsx	Curriculum selector, requirement cards, JSON view, mapping search, pagination, tag filtering, and tag inventory.
apps/api/src/routes/adminRoutes.ts	Admin-only read routes for the curriculum catalogue, latest RequirementSet, module mappings, and available tags.
apps/api/src/services/adminCurriculumService.ts	Aggregates version counts and mapping counts, loads the latest ruleset, searches modules, and groups the complete tag inventory.
apps/api/src/middleware/auth.ts	requireAdmin rejects non-administrator accounts before any administration route runs.
apps/api/src/models/RequirementSet.ts	Stores versioned curriculum rules selected by programme and cohort.
apps/api/src/models/ModuleRequirementTags.ts	Stores programme/cohort/module-specific classification tags.
apps/api/src/models/Module.ts	Provides the searchable NUS module catalogue joined to curriculum mappings.

Implementation Philosophy

The view workflow is deliberately read-only and authoritative. Curriculum rules and module mappings are loaded through dedicated admin endpoints guarded by requireAuth and requireAdmin, while the frontend only stores navigation state such as the selected curriculum, active tab, search, filter, and page. This separation prevents a viewing action from becoming an accidental database mutation and makes the latest published version the clear source of truth.

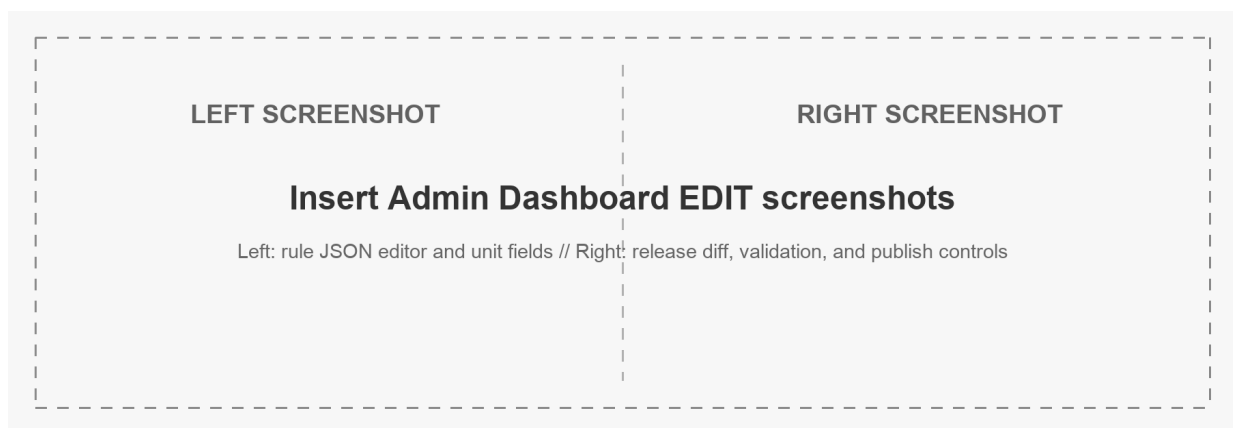
Module metadata and curriculum meaning remain separate. Module stores general NUS information, while ModuleRequirementTags classifies the same module differently for each programme and cohort. The dashboard preserves that model by searching the shared module catalogue first and then presenting the mapping in the selected curriculum context. It therefore scales without copying module title, description, prerequisite, or faculty data into every ruleset.

9. Admin Dashboard: Edit, Validate, Diff, and Publish

Description

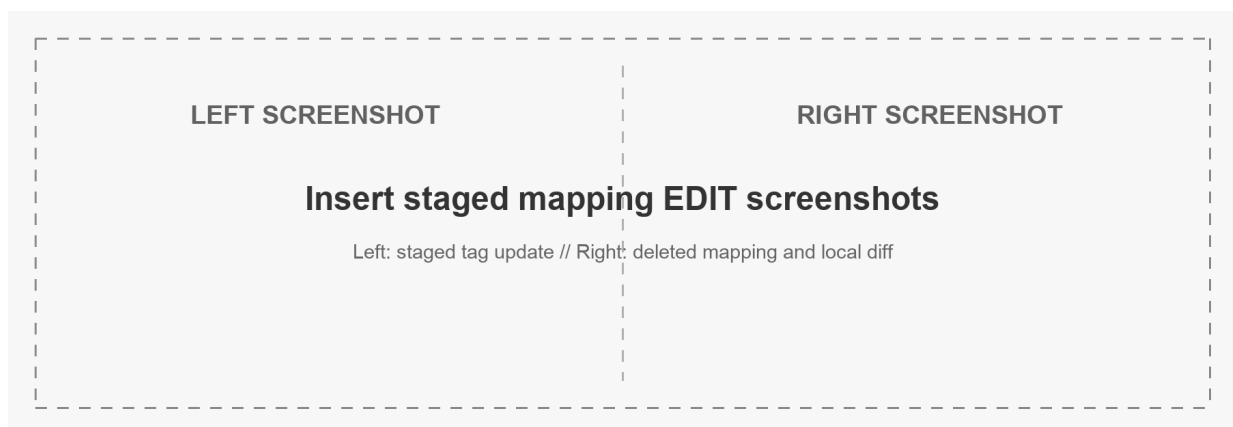
The edit workflow lets an administrator change a curriculum's total units, source note, rule order, and individual requirement rules directly in the web application. Each rule can be edited as formatted JSON. The editor parses the text and validates it with RequirementRuleSchema before replacing the in-memory rule, so temporarily incomplete JSON remains local to the editor and invalid rule objects never enter the curriculum draft. Required units and every rule-specific threshold remain part of that same canonical JSON representation.

New rules are added through a rule id, label, and rule-type selector. Selecting a type creates the matching JSON template instead of starting from an unstructured empty object. For example, a module-list rule starts with requiredModules, a capped-units-from-tags rule starts with acceptedTags and tagCaps, and the structured rule types start with the threshold arrays and unit fields needed by their evaluator. The administrator can then edit, reorder, duplicate, or remove rules while preserving the explicit priority order used during allocation.



Left: editing one requirement's JSON and unit fields in the web application // Right: rule and mapping diff, validation state, next version, and publish controls.

Module mappings use a staged editing model. An administrator searches for a module, edits its comma-separated tag keys, and stages the result as an AdminModuleTagChange. A mapping can also be staged for deletion. Re-entering the published tags removes the staged change, and the complete staged set can be cleared without affecting the database. Rule edits and mapping edits are persisted in administrator-specific localStorage so that changing tabs or returning to the dashboard does not lose an unfinished draft.



Left: a locally staged module-tag update // Right: a staged mapping deletion with the corresponding diff before publication.

The release sidebar continuously produces a diff against the latest published RequirementSet. It reports total-unit and source-note changes, rules added, changed, or removed, ordinary module-tag changes, mapping deletions, and the next version number. Validation then checks the entire draft rather than a single field: shared Zod schemas, unique rule ids, required fields for each rule type, known module references, duplicate tag changes, referenced tags, declared unit totals, the latest base version, and regression previews against up to five saved plans for the same programme and cohort.

All edits remain local until the exact current draft has passed validation and the administrator confirms publication. Publishing creates a new RequirementSet version and applies all staged mapping upserts or deletions in one MongoDB transaction. The previous RequirementSet remains stored, and a baseVersion check rejects stale drafts if another version was published after the draft was loaded. After success, React Query invalidates the administration, requirements, module-tag, and evaluation caches so users receive the new curriculum data.

Important file	Purpose
apps/web/src/pages/AdministratorPage.tsx	Rule JSON editor, type-tagged rule templates, local draft persistence, staged mapping changes, diff summary, validation fingerprint, and publish controls.
packages/shared/src/index.ts	RequirementRuleSchema, RequirementRuleTypeSchema, AdminModuleTagChangeSchema, and AdminCurriculumDraftSchema.
apps/api/src/services/adminCurriculumService.ts	Semantic validation, tag coverage checks, sample-plan regression previews, base-version checks, transactional publication, and mapping bulk writes.
apps/api/src/routes/adminRoutes.ts	POST /validate and POST /publish endpoints behind administrator authorization.
apps/api/src/models/RequirementSet.ts	Preserves every published version under the unique programme/cohort/version identity.
apps/api/src/models/ModuleRequirementTags.ts	Receives staged mapping upserts and deletions only during publication.
packages/rules-engine/src/evaluatePlan.ts	Evaluates saved sample plans against the proposed RequirementSet during validation.

Implementation Philosophy

Curriculum maintenance follows a draft-validate-publish workflow rather than in-place editing. Local drafts make experimentation safe, the diff makes the release understandable, and versioned publication keeps prior rulesets available for diagnosis. The validated fingerprint prevents the administrator from validating one draft, changing it, and publishing a different draft without another validation pass.

Rules and module mappings are published atomically because they describe one curriculum interpretation. Creating a RequirementSet version without its matching tag changes could make valid rules evaluate incorrectly, while changing tags without the intended rules could silently reallocate modules. A MongoDB transaction and optimistic baseVersion check prevent those split-brain and lost-update states.

Rule-type templates are intentionally explicit. Each evaluator branch expects a different combination of fields, so selecting a rule type creates a valid structural starting point and exposes the resulting JSON for review. This keeps the admin interface flexible enough for advanced curricula without hiding the data model behind dozens of one-off form screens.

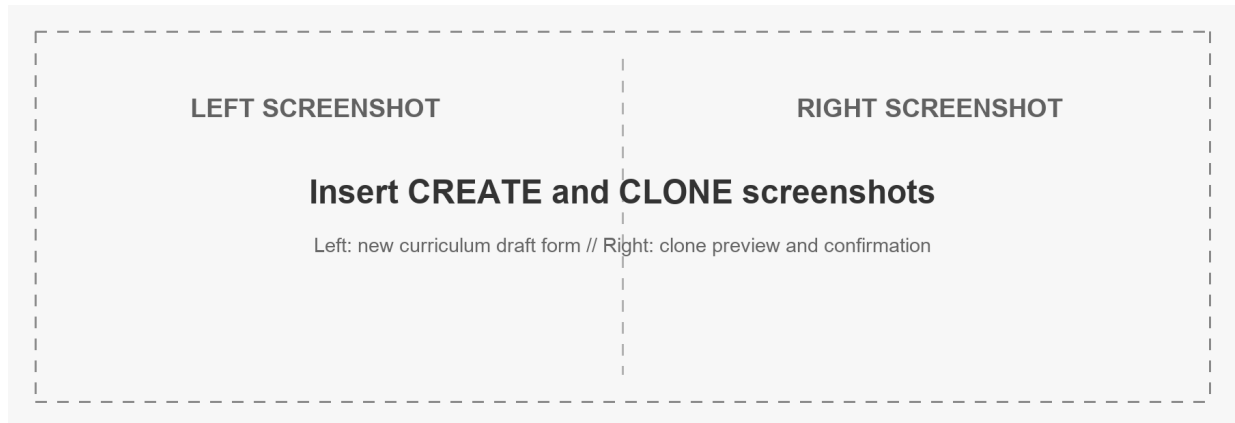
10. Admin Dashboard: Create and Clone Curricula

Description

An administrator can create an entirely new curriculum by selecting a programme identity and entering a cohort. CohortSchema requires the AY2025/26 format and verifies that the end year is exactly the year after the start year. The dashboard also checks the published catalogue so an existing programme/cohort pair cannot be recreated accidentally. A new curriculum begins as a local baseVersion 0 draft with a 160-unit default, an empty source note, one starter rule, and no module-tag changes; it becomes visible to students only after full validation and publication as version 1.

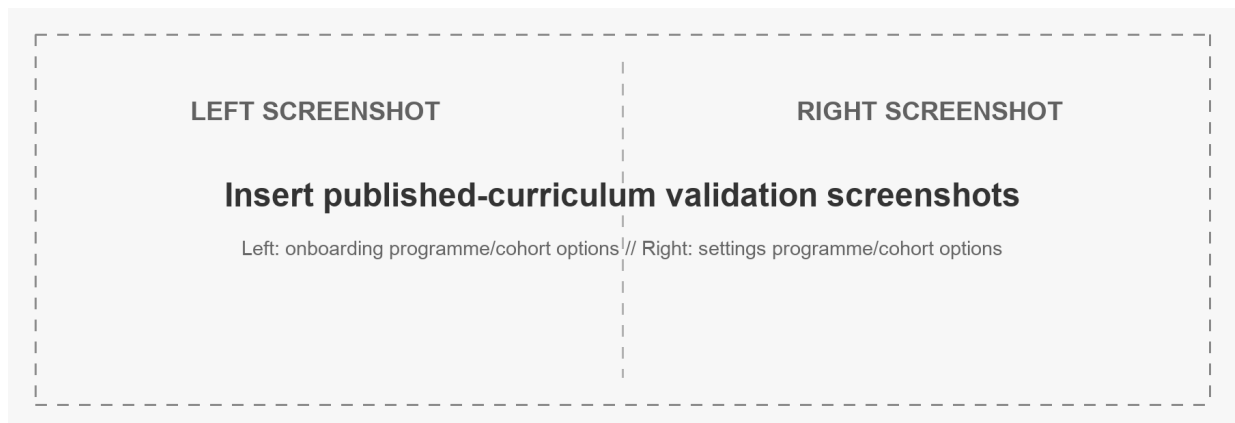
The clone workflow is optimised for a new academic year whose requirements have not changed. The administrator selects a complete source curriculum and a target programme/cohort, then previews the operation. The preview shows the source version, total units, rule count, tag count, and source note; checks whether the target already has any RequirementSet version or module mappings; preserves references to historical modules while warning about modules missing from the current catalogue; and reports tags referenced by rules but absent from the source mappings.

If the target is clear, cloning copies the entire rule array, source note, total units, and all module mappings into a transaction and creates the target at version 1. This is a complete curriculum copy rather than a frontend convenience: both the RequirementSet and ModuleRequirementTags records receive the new programme/cohort identity, so the cloned cohort is immediately independent and can be edited and versioned later.



Left: creating a new local curriculum draft with validated programme and cohort // Right: clone preview showing source statistics, conflicts, warnings, and the target version.

Onboarding and Settings use the published requirement-set catalogue as their allowed programme and cohort options. The programme dropdown is derived from programmes that have at least one published ruleset, and the cohort dropdown is filtered to published cohorts for the selected programme. A user therefore cannot choose an unsupported curriculum through these pages, while a newly published or cloned curriculum becomes selectable without hardcoding another frontend option.



Left: onboarding only offers published programme/cohort combinations // Right: Settings applies the same curriculum-backed validation before changing all plans.

Important file	Purpose
apps/web/src/pages/AdministratorPage.tsx	New-curriculum form, cohort validation feedback, clone target selection, preview, confirmation, and local baseVersion 0 drafts.
packages/shared/src/index.ts	CohortSchema validates consecutive academic years; AdminCloneCurriculumSchema prevents cloning onto the same curriculum.

Important file	Purpose
apps/api/src/services/adminCurriculumService.ts	Clone preview, conflict detection, warning generation, and transactional copying of rules and module mappings.
apps/api/src/services/requirementService.ts	Lists published programme/cohort pairs used as the student-facing curriculum catalogue.
apps/web/src/pages/OnboardingPage.tsx	Builds programme and cohort choices only from published rulesets.
apps/web/src/pages/SettingsPage.tsx	Restricts academic profile changes to published programme/cohort combinations.
apps/api/src/services/profileService.ts	Applies the selected published curriculum identity to the profile and every owned plan.

Implementation Philosophy

Creation and cloning share the same publication boundary. A new identifier is not treated as supported merely because it appears in the programme enum; it becomes a usable curriculum only when a valid RequirementSet exists. This makes the database catalogue a capability registry for onboarding, settings, evaluation, and administration.

Cloning is intentionally exact and conservative. If requirements are unchanged, copying the complete versioned curriculum is safer than re-entering rules and mappings. If requirements later diverge, the administrator edits the independent target draft and publishes a new version. Target conflict checks and transactions prevent a clone from overwriting an existing cohort or leaving a half-created curriculum.

11. Scaling to Five School of Computing Curricula

Description

Milestone 3 scales The CS Plan from one Computer Science ruleset to five live programme identities: Computer Science, Business Analytics, Business Artificial Intelligence Systems, Computer Science and Mathematics Double Major, and Business Analytics and Economics Double Degree. Computer Science is available for AY2025/26 and AY2026/27, so the five programmes are represented by six published programme/cohort rulesets. Each curriculum has its own total-unit target, official source note, ordered requirement rules, and module-tag mappings.

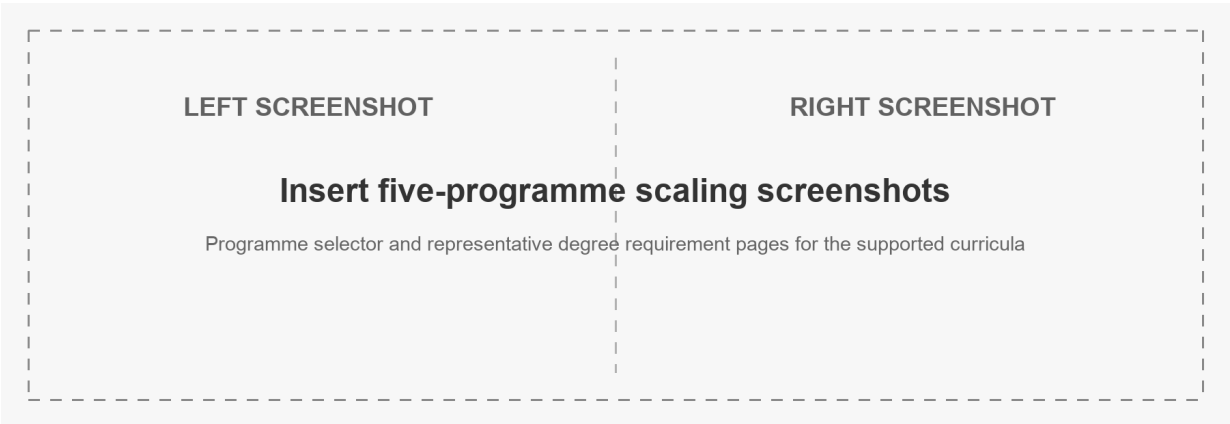
The rulesets were created by translating the published curriculum structures into RequirementSet data and mapping relevant module codes into programme/cohort-specific tags. Existing rule types were reused wherever they represented the published requirement accurately: module-list, units-from-tags, capped-units-from-tags, structured-idcd, structured-breadth-depth, and residual-units continue to serve multiple programmes. New evaluator branches were added only when a

genuinely new structure could not be expressed safely with existing fields, including module-choice, structured-programme-electives, structured-industry-experience, and structured-ddp-honours-pathway.

Business Analytics and Business Artificial Intelligence Systems reuse the same general University Pillars, Computing Ethics, ID/CD, programme-elective, industry-experience, and unrestricted-elective concepts while supplying different core modules, tags, thresholds, and dissertation or internship alternatives. Their mapping generators also classify broad families of NUS modules, then assign unmapped catalogue modules to unrestricted electives where the curriculum permits.

Computer Science and Mathematics Double Major reuses the AY2025/26 Computer Science ruleset and module mappings as its base. It replaces the Mathematics and Sciences rule with the second-major categories and caps, removes conflicting UE or CD classifications from Mathematics modules, and reduces the unrestricted-elective target. This demonstrates that a related curriculum can reuse stable rules and alter only the parts that differ.

A double major or double degree is modelled as one curriculum identity, not as two independently selected programmes. The Business Analytics and Economics Double Degree therefore has one dedicated requirement set that encodes shared courses, both discipline cores, programme electives, and the integrated-thesis or internship honours pathways. Treating the published course as one curriculum avoids ambiguous merging, duplicate allocation, and mismatched total-unit targets.



Five supported programme identities: Computer Science, Business Analytics, Business Artificial Intelligence Systems, Computer Science and Mathematics Double Major, and Business Analytics and Economics Double Degree.

Important file	Purpose
packages/data/src/computerScienceData.ts	Computer Science AY2025/26 and AY2026/27 rulesets, focus areas, mappings, and generated common classifications.
packages/data/src/businessAnalyticsData.ts	Business Analytics core, statistics alternatives, programme electives, industry pathways, and mappings.

Important file	Purpose
<code>packages/data/src/baisData.ts</code>	Business Artificial Intelligence Systems core, programme electives, industry pathways, and mappings.
<code>packages/data/src/csMathDoubleMajData.ts</code>	Reuses Computer Science rules and mappings, then replaces Mathematics requirements and adjusts unrestricted electives.
<code>packages/data/src/bzaEconsDdpData.ts</code>	Dedicated Business Analytics and Economics Double Degree curriculum and honours-pathway mappings.
<code>packages/rules-engine/src/evaluatePlan.ts</code>	Shared evaluator branches reused across all five programme identities.
<code>apps/api/src/upsertCurricula.ts</code>	Upserts the current curricula and complete generated module mappings into MongoDB.
<code>apps/api/src/tests/*Data.test.ts</code>	Curriculum-specific data regression tests for totals, rules, official constraints, and mappings.

Implementation Philosophy

Scaling is data-first. React pages and Express routes do not branch on every degree; they request the selected programme/cohort RequirementSet and mapping lookup, then pass both to the same evaluator. The user interface can therefore support a newly published curriculum without a new planner page, while genuinely new academic logic is introduced as a named, validated rule type rather than programme-specific conditionals.

Curriculum identities match the choices published to students. A combined course can reuse rules and mappings internally, but it remains one selectable programme with one allocation model and one total-unit target. This prevents the planner from naively combining two standalone programmes whose overlapping modules, common curriculum, honours routes, or unrestricted-elective rules may interact.

Data and Application Flow

1. The user signs in through Google OAuth and receives an HTTP-only `tcp_session` cookie.
2. The frontend calls `/api/me`. If no profile exists, it routes the user to onboarding.
3. Onboarding saves `StudentProfile` and creates or reconciles the user's owned plans. The profile stores programme, cohort, starting semester, current semester, graduation semester, `primaryPlanId`, and `planOrder`.

4. The planner loads the ordered plan list, the selected plan, module search results, requirement definitions, and module requirement tags for the plan's programme and cohort.
5. When the user edits modules, placeholders, plan names, plan order, grades, or S/U flags, the frontend sends the full Plan object through the owner-scoped plan API.
6. The backend verifies ownership, validates the Plan schema, and saves the updated semesters. Profile updates also sanitise primaryPlanId and planOrder so a user cannot point to another user's plans.
7. Evaluation loads modules, the latest requirement set, and module tags, then calls evaluatePlan from the rules engine. Prerequisite warnings are added after the degree requirement evaluation.
8. The frontend displays progress rows and advisory warnings, caching the latest evaluation result locally for smoother user experience.
9. For sharing, export builds a PlanExport snapshot, encodes it into a tcp1_ payload or import link, and import creates a new user-owned plan from that snapshot instead of overwriting an existing plan.

Milestone 3 Curriculum Administration Flow

An authenticated administrator opens /administrator. The frontend verifies the User.role returned by /api/me, redirects non-administrators to the planner, and enables all /api/admin calls only for an administrator.

The dashboard loads the curriculum catalogue, the latest RequirementSet for the selected programme/cohort, the current module mappings, and the complete tag inventory. Curriculum, editor, and search state are namespaced to the administrator and curriculum in localStorage.

Rule JSON, total units, source note, order, and staged AdminModuleTagChange objects are edited locally. The frontend compares this draft with the latest published ruleset and displays rule additions, changes, removals, mapping changes, mapping deletions, and the next version.

Validation parses the AdminCurriculumDraft through shared Zod schemas, checks rule-specific semantics and module references, applies staged mappings to an in-memory lookup, warns about

missing tags or mismatched unit totals, checks baseVersion, and evaluates up to five recent saved plans for that curriculum.

Publication revalidates the draft, starts a MongoDB session, confirms that baseVersion is still current, inserts version baseVersion + 1, and bulk-writes the staged mapping updates or deletions in the same transaction. The frontend then clears the local draft and invalidates curriculum-dependent caches.

Cloning follows a separate preview path. The backend verifies that the target has no RequirementSet or mappings, reports retained historical modules and unmapped tags, and then transactionally copies the complete source curriculum into target version 1.

Student onboarding and settings request the published curriculum catalogue. The selected programme filters the allowed cohorts, so only an existing RequirementSet can be written to StudentProfile and propagated to the user's plans.

Technology Stack

Frontend

The web application is written in TypeScript 5.8 with React 19.1 and React DOM. Vite 6.3 provides the development server and production bundle, while React Router 7.6 defines login, onboarding, planner, GPA, requirements, settings, and administrator routes. TanStack React Query 5.81 manages server state, caching, mutations, and invalidation across profile, plan, evaluation, curriculum, and administration workflows.

Forms use React Hook Form 7.58 with @hookform/resolvers and shared Zod 3.25 schemas. Tailwind CSS 3.4, PostCSS, Autoprefixer, clsx, and tailwind-merge implement the responsive dark interface and reusable class composition. Lucide React supplies the interface icon set. Low-risk client persistence uses localStorage for evaluation caches, dismissed warning state, and administrator drafts or editor state; authoritative academic data remains on the API.

Backend

The API is a Node.js TypeScript application built with Express 5.1. Express routers separate authentication, current-user, profile, plan, module, requirement, and administration endpoints. Zod validates route bodies and shared domain objects. Mongoose 8.16 provides models, indexes, queries, aggregation pipelines, bulk writes, sessions, and transactions. The API uses JSON Web Tokens, cookie-parser, and HTTP-only tcp_session cookies for seven-day sessions; CORS is configured for credentialed requests from the web client.

Google OAuth is implemented in the backend without coupling Google tokens to academic records. dotenv loads deployment configuration such as MONGODB_URI, CLIENT_URL, SERVER_URL, SESSION_SECRET, GOOGLE_CLIENT_ID, and GOOGLE_CLIENT_SECRET. The backend normalises NUSMods catalogue data, serves owner-scoped plans, composes requirement evaluation inputs, and enforces the administrator role before curriculum operations.

Database and Domain Packages

MongoDB stores User, StudentProfile, Plan, Module, ModuleRequirementTags, and RequirementSet collections. Plans embed semester and plan-item snapshots because planner and GPA updates operate on the complete scenario. RequirementSet documents are append-only by version, while programme/cohort/module mappings are updated transactionally with a publication. Compound unique indexes protect module catalogue identities, mapping identities, and curriculum versions.

The npm-workspace monorepo separates packages/shared for Zod schemas, types, programme labels, GPA constants, semester helpers, and S/U eligibility; packages/data for the five supported curricula and generated mappings; and packages/rules-engine for pure requirement allocation and prerequisite evaluation. This keeps React, Express, data definition, and curriculum evaluation independently testable while sharing one TypeScript contract.

Tooling, Testing, and Deployment

npm workspaces coordinate builds and tests, tsx runs TypeScript development and data scripts, and concurrently starts the web and API applications together. Vitest 3.2 runs API, shared-domain, curriculum-data, and rules-engine tests; Supertest is available for HTTP-level API workflow coverage. TypeScript project configurations provide package builds and no-emit type checking. The frontend includes a Vercel single-page-application rewrite, while the API runs from apps/api with environment-based database and OAuth configuration.

Software Engineering Practices

TypeScript Monorepo Boundaries

The CS Plan is structured as a TypeScript monorepo with separate packages for the frontend, backend, shared schemas, seeded data, and the rules engine. This supports the login/onboarding, planner, GPA tracker, multiple plans, import/export, and requirements features because each layer has a clear responsibility: React handles interaction, Express handles authenticated APIs, MongoDB stores state, shared schemas define contracts, and the rules engine evaluates curriculum logic.

Shared Zod Validation

Core data shapes such as Plan, StudentProfile, Module, RequirementSet, RequirementRule, ModuleRequirementTags, ModuleGrade, and PlanExport are defined in packages/shared/src/index.ts using Zod. This is used directly by profile updates, plan updates, import/export, and requirement evaluation so frontend and backend agree on the same structure.

Data-Driven Requirement Evaluation

Degree Requirement Fulfillment is implemented through packages/rules-engine instead of route-level or component-level conditionals. Requirements are represented as versioned data, while the evaluator handles allocation, placeholders, tag matching, structured ID/CD, breadth/depth rules, programme electives, industry experience, double-degree honours pathways, residual

units, and warnings. This practice enabled Milestone 3 to add five programme identities and another cohort primarily through new requirement data and deliberate reusable rule types rather than new UI branches.

Owner-Scoped Backend Services

Plan reads, updates, deletes, exports, imports, and evaluations go through owner-scoped service functions such as `getOwnedPlan`, `updateOwnedPlan`, `deleteOwnedPlan`, `exportOwnedPlan`, and `evaluateOwnedPlan`. This protects user plans across Login, Multiple Plans, GPA tracking, and plan sharing because the backend checks ownership before returning or mutating a plan.

Snapshot-Based Sharing

The Plan Import / Export feature uses schema-versioned snapshots instead of live shared records. The receiver imports a copy under their own user id, while MongoDB ids are removed from the sender's plan. This keeps sharing simple, avoids accidental cross-user writes, and gives the project a clear path for future backwards-compatible payload versions.

UI State and Cache Discipline

The frontend uses React Query for server state and `localStorage` only for low-risk client caches such as the latest evaluation result and dismissed warning keys. Planner edits invalidate or refresh the relevant plan and evaluation data, while GPA edits update the local plan cache optimistically and fall back to refetching if saving fails.

Role-Based Access Control

The administrator feature applies the same authorization rule at both interface and API boundaries. `AdministratorPage` redirects a student account, while every `/api/admin` route passes through `requireAuth` and `requireAdmin`. This practice protects curriculum viewing, validation, publication, and cloning even if a non-administrator calls the API directly.

Draft, Diff, Validation, and Publish Workflow

Curriculum editing is split into explicit states: published data, local draft, validated draft, and published next version. The feature uses administrator-and-curriculum-scoped `localStorage`, a human-readable diff, and a JSON fingerprint of the validated draft. Publication is disabled whenever the draft changes after validation. This practice links directly to rule editing, staged module mappings, deletions, and new-curriculum creation.

Transactional Versioning and Optimistic Concurrency

`RequirementSet` publication and `ModuleRequirementTags` changes run in one MongoDB transaction. A `baseVersion` comparison is performed during validation and again inside the transaction, so concurrent administrators cannot silently overwrite one another. Every successful release creates a new `RequirementSet` version, while cloning creates a conflict-checked version 1 for a new curriculum identity.

Rule-Type Templates and Deliberate Reuse

The administrator's rule-type selector is tagged to createRequirementRule, which generates the fields required by the chosen evaluator branch. Curriculum data reuses stable rule types and adds a new type only when a published constraint cannot be represented safely. This practice supports the JSON editor, five-programme scaling, and consistent semantic validation.

Curriculum-Catalogue-Driven Interfaces

Onboarding and Settings derive programme and cohort choices from published RequirementSet identities. The same catalogue is used by the administrator selector and student-facing profile flows. This removes duplicated option lists and ensures that users can only choose a curriculum that the evaluator can load.

Programme-Specific Module Classification

ModuleRequirementTags keeps shared NUS module metadata separate from curriculum-specific meaning. Generated and explicit mappings can therefore classify one module differently across Computer Science, Business Analytics, BAIS, a double major, and a double degree. This practice supports requirement evaluation, module-tag administration, and future expansion without duplicating the module catalogue.

Pure Domain Logic and Regression Fixtures

Requirement evaluation, prerequisite checking, S/U eligibility, schema parsing, and admin semantic checks are deterministic functions with focused Vitest coverage. Curriculum-specific fixture tests assert published unit totals, rule structures, mappings, and complex pathways. This practice makes ruleset reuse and administration changes safer because regressions are caught before data is deployed.

Testing

The Milestone 3 repository test suite contains 83 passing Vitest tests across 10 test files. It covers the rules engine, S/U eligibility, NUSMods normalisation, authentication and administrator guards, administrator draft contracts and semantic validation, and regression checks for all five supported programme identities. The suite is run across npm workspaces with npm test.

Unit and Domain Logic Testing

packages/rules-engine/src/tests/evaluatePlan.test.ts contains 48 tests for allocation, module lists, module choices, tag caps, placeholders, ID/CD constraints, breadth and depth, programme electives, industry experience, double-degree honours routes, unrestricted-elective overflow, programme-specific mappings, unknown modules, and prerequisite-tree evaluation. apps/api/src/tests/suEligibility.test.ts adds seven GPA-page policy tests for module levels, prerequisite-bearing Level-2000 modules, published exceptions, language modules, explicit exclusions, and unknown module codes.

`apps/api/src/tests/nusmodsImportService.test.ts` verifies module-code, title, units, department, faculty, description, prerequisite text, and `prereqTree` normalisation, including invalid module-credit fallback and detail-payload merging. These tests isolate deterministic calculations and transformations before they are used by API routes or React pages.

Authorization, Schema, and Service Contract Testing

`apps/api/src/tests/authMiddleware.test.ts` verifies unauthenticated rejection, student rejection from administrator routes, and administrator access. `apps/api/src/tests/adminCurriculumService.test.ts` verifies consecutive academic-year cohorts, normalised and de-duplicated tags, staged mapping deletion, clone self-target rejection, valid drafts, duplicate rule ids, missing rule fields, unknown module references, duplicate mapping changes, and the controlled exception for historical rule modules during clone validation.

Integration Testing

API integration testing verifies the full route-service-schema path for profile onboarding, multiple plan creation and ordering, owner-scoped updates and deletion, plan import/export, requirement evaluation, settings updates, administrator validation and publication, and curriculum cloning. The checks confirm authentication and administrator authorization, Zod request validation, service composition, persistence semantics, and the response shapes consumed by React Query.

Automated User Interface Testing

Frontend workflow tests exercise the planner actions proposed in the Milestone 2 README: adding a module, adding a placeholder, moving an item between semesters, renaming and reordering plans, changing the primary plan, importing and exporting with or without grades, deleting a plan, displaying prerequisite warnings, and refreshing degree progress. Milestone 3 workflows also cover viewing a curriculum, editing rule JSON, adding a type-templated rule, staging or deleting a module mapping, validating a draft, publishing a version, creating a curriculum, cloning a cohort, and restricting onboarding or settings to published curricula.

Curriculum Data and Regression Testing

Fifteen curriculum-data tests cover Computer Science, Business Analytics, Business Artificial Intelligence Systems, Computer Science and Mathematics Double Major, and Business Analytics and Economics Double Degree. They assert the supported cohort, total units, rule types, official core or elective structures, industry and honours pathways, generated classifications, and representative mappings. The evaluator's fixture plans then catch broken allocation logic when a rule type or mapping changes.

GPA and S/U Testing

GPA tracker testing covers grade-point conversion, pre-S/U and post-S/U GPA, semester GPA, current GPA before the current semester, entire-plan GPA, S/U-only grades, CS grades, zero graded units, placeholders, and missing grades. The S/U eligibility unit tests additionally confirm

that the checkbox logic follows module level, NUS-course prerequisites, published Level-2000 exceptions, language-course eligibility, and explicit exclusions.

User Testing

After scaling to the five currently available programme identities, the team deployed all supported curricula live and invited 20 friends studying one of these programmes to use the application with their own academic plans. The participants tested onboarding, programme and cohort selection, module planning, requirement progress, GPA tracking, S/U decisions, and the general usability of the deployed workflow. Their feedback led to small interface and validation improvements before the Milestone 3 release.

One concrete issue found during this round was that the GPA page originally allowed any module to be marked S/U. Under NUS policy, only eligible modules can be S/Ued. The application now loads module metadata and calls `getSuEligibility` using the student's cohort. The logic considers course level, whether a Level-2000 course has an NUS-course prerequisite, published exceptions, language-course eligibility, and explicit exclusions. Ineligible modules have a disabled S/U control and an explanatory tooltip. The new policy behaviour is protected by seven dedicated unit tests.

Future Plans

1. Scale from School of Computing to the Entire NUS Population

The next expansion should introduce a faculty-owned curriculum registry. `RequirementSet` metadata would gain a `homeFaculty` classification and optional `jointFaculty` metadata, the administrator catalogue would be filterable by faculty, and onboarding would follow Faculty -> Programme -> Cohort. Shared NUS-wide requirements could still reuse common rule types, while each home faculty would own its source notes, mappings, and regression fixtures. A tangible first phase is to add the metadata and administration filters, onboard at least one complete pilot curriculum outside SoC, validate it against representative plans, and then repeat the same publication workflow faculty by faculty until the full NUS population is covered.

This classification also handles joint programmes cleanly. A double degree remains one curriculum identity, but its primary home faculty and collaborating faculty can be recorded for discovery and maintenance responsibility. The planner and rules engine continue to consume programme/cohort data, while the new faculty layer improves navigation, governance, and scaling.

2. Persisted Curriculum Audit, Review, and Rollback

The current administrator workflow has local drafts, validation, diffs, transactions, and versioned `RequirementSet` documents. The next tangible step is a persisted audit and approval layer: store draft ownership, source references, change summaries, validation results, reviewer approval, publication timestamp, and the `RequirementSet` version created by the release. High-impact changes could require a second administrator before publication, and any prior version could be

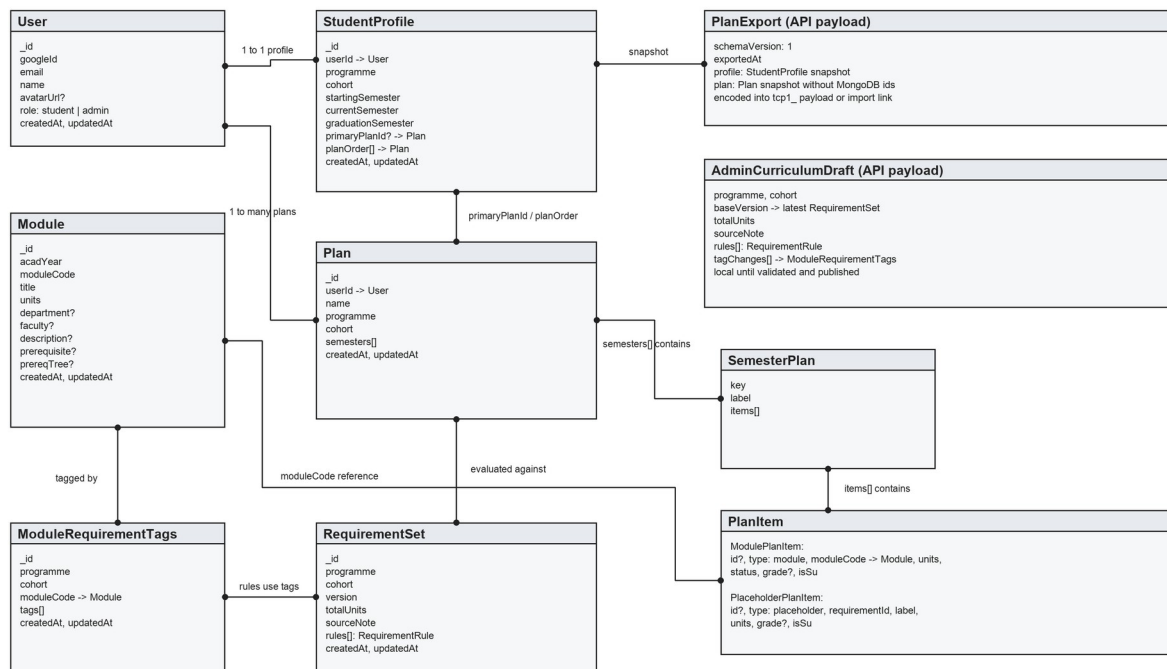
selected as the source of a rollback draft. This would turn technical versioning into an operational curriculum-governance process suitable for multiple faculty maintainers.

3. Constraint-Aware Semester Recommendations

The planner should progress from showing missing requirements to proposing feasible ways to satisfy them. A recommendation service can combine the current RequirementSet, allocation result, prerequisite trees, historical semester offerings, timetable and examination clashes, and a user-selected workload range. It would rank module combinations that close the most requirement gaps without violating prerequisite order or semester constraints, present the reasons for each recommendation, and let the student preview the suggested changes in a new plan before accepting them. A first release can target one empty semester and a single active plan, then expand to multi-semester optimisation.

Database Schema Appendix

This appendix describes the current Milestone 3 MongoDB models, embedded plan shapes, versioned curriculum records, and non-persisted API payloads. Mongoose defines persistence and indexes in `apps/api`, while Zod schemas in `packages/shared` validate API contracts, curriculum rules, administrator drafts, exports, and the discriminated plan-item union. Plan semesters and RequirementSet rules are stored as mixed embedded data in MongoDB because the shared Zod schemas are the canonical shape.



Milestone 3 Entity Relationship Diagram (ERD)

MongoDB Collections and API Payloads

Model or payload	Fields and design notes
User	_id, googleId, email, name, avatarUrl?, role, createdAt, updatedAt. role is student or admin; Google identity remains separate from academic state.
StudentProfile	_id, userId, programme, cohort, startingSemester, currentSemester, graduationSemester, primaryPlanId?, planOrder[], createdAt, updatedAt. One profile owns the account-wide academic context and plan ordering.
Plan	_id, userId, name, programme, cohort, semesters[], createdAt, updatedAt. A user can own many independent scenarios; grades and S/U choices are stored inside each plan's items.
Module	_id, acadYear, moduleCode, title, units, department?, faculty?, description?, prerequisite?, prereqTree?, createdAt, updatedAt. This is the shared NUS module catalogue.
ModuleRequirementTags	_id, programme, cohort, moduleCode, tags[], createdAt, updatedAt. This maps a shared module to curriculum-specific classifications.
RequirementSet	_id, programme, cohort, version, totalUnits, sourceNote, rules[], createdAt, updatedAt. Publication inserts a new version rather than overwriting the previous ruleset.
PlanExport API payload	schemaVersion, exportedAt, profile snapshot, and plan snapshot without MongoDB ids. It is encoded as a tcp1_ payload or import link and creates an independent imported Plan.
AdminCurriculumDraft API payload	programme, cohort, baseVersion, totalUnits, sourceNote, rules[], tagChanges[]. It is local until validation and is never stored as a MongoDB collection.

Model or payload	Fields and design notes
AdminCloneCurriculum API payload	sourceProgramme, sourceCohort, targetProgramme, targetCohort. Preview checks conflicts and warnings; clone creates target version 1 and copies all module mappings.

Embedded Plan and Administrator Object Shapes

Embedded object	Shape
SemesterPlan	key, label, items[]. key is IBLOC or Y1S1 through Y5S2 within the profile's starting/graduation range.
ModulePlanItem	id?, type: module, moduleCode, units, status, grade?, isSu. status is completed, current, or planned.
PlaceholderPlanItem	id?, type: placeholder, requirementId, label, units, grade?, isSu. It reserves requirement units without pretending to be a real module.
AdminModuleTagChange	moduleCode, tags[], deleteMapping?. Tags are trimmed, lower-cased, format-validated, and de-duplicated. A deletion must have an empty tag list.

Indexes and Relations

Relation or index	Purpose
User.googleId and User.email are unique	Prevents duplicate accounts for the same Google identity or email.
StudentProfile.userId is unique	Enforces one academic profile per User.
StudentProfile.primaryPlanId and planOrder[] reference Plan	Stores the default plan and display order; profile services sanitise both fields to owned plan ids.
Plan.userId index	Supports efficient owner-scoped plan listing and mutation.
Module unique acadYear + moduleCode	Prevents duplicate catalogue entries for one module in the same academic year.

Relation or index	Purpose
ModuleRequirementTags unique programme + cohort + moduleCode	Allows exactly one mapping document per module for each curriculum identity.
RequirementSet unique programme + cohort + version	Preserves append-only curriculum versions while rejecting duplicate version numbers.
Plan.programme + cohort resolves RequirementSet and mappings	Evaluation loads the latest matching RequirementSet and programme/cohort-specific tag lookup.

Requirement Rule Shape

Rule object	Supported fields
RequirementSet	programme, cohort, version, totalUnits, sourceNote, rules[].
Common RequirementRule fields	id, label, type, requiredUnits?, requiredModules?, moduleOptions?, acceptedTags?, acceptedPlaceholders?, tagCaps?, advisory?.
Structured ID/CD and breadth/depth fields	idTags?, cdTags?, requiredIdMinCourses?, allowedCdMaxCourses?, focusAreas?, focus-area counts, Level-4000 units, industry min/max, allowed prefixes, CP cap, industryTags?, dissertationTags?.
Programme elective and industry fields	requiredMinCourses?, requiredLevel4000MinCourses?, requiredPrefixMinCourses?, requiredPrefixes?, internshipFoundationTags?, secondInternshipTags?, supplementaryTags?, requiredFoundationUnits?, requiredCompanionUnits?, tagUnitOverrides?.
Double-degree honours fields	integratedThesisTags?, economicsElectiveTags?, economicsLevel4000Tags?, integratedThesisUnits?, integratedEconomicsUnits?, integratedEconomicsLevel4000Units?, internshipEconomicsUnits?, internshipEconomicsLevel4000Units?, internshipPathwayRequiredUnits?.

Rule object	Supported fields
Rule types	module-list, module-choice, units-from-tags, capped-units-from-tags, placeholder-units, combined-units, structured-idcd, structured-breadth-depth, structured-programme-electives, structured-industry-experience, structured-ddp-honours-pathway, residual-units.

Administrator Draft and Publication Semantics

AdminCurriculumDraft is a non-persisted API contract with baseVersion. Validation overlays tagChanges on the current mapping lookup, verifies rule semantics and module references, compares baseVersion with the latest RequirementSet, and evaluates up to five recent plans. Publication repeats the version check inside a MongoDB transaction, inserts the next RequirementSet version, and applies mapping upserts or deletions atomically.

Clone preview is also non-persisted. It checks that the target RequirementSet and mapping namespace are empty, retains historical rule references with warnings, and reports source statistics. A successful clone creates target version 1 and copies every source ModuleRequirementTags record in the same transaction.

Why These Relations Matter

The schema keeps identity, academic profile, planning scenarios, shared module metadata, curriculum-specific module interpretation, and versioned curriculum rules separate. Each category changes for a different reason: users update timelines, plans branch into alternatives, NUSMods updates catalogue metadata, and faculties publish new curriculum versions.

Plan embeds SemesterPlan and PlanItem because planner and GPA screens usually read and save the complete scenario. Module and RequirementSet are shared reference data. ModuleRequirementTags bridges a module's stable identity to a curriculum's meaning. PlanExport, AdminCurriculumDraft, and AdminCloneCurriculum remain API payloads rather than collections so sharing creates independent copies and administration publishes only validated versioned records.

Versioned RequirementSet documents plus transactional mapping publication provide curriculum history without duplicating the shared module catalogue. The programme/cohort identity also lets onboarding, settings, plan evaluation, administrator selection, and test fixtures resolve the same curriculum consistently.